BLOCKCHAIN EXPERT ROADMAP

DAY 2

Wallets, Keys & Digital Signatures

Complete 60-Minute Beginner Lecture

Audience	Complete beginners
Duration	60 minutes
Career Focus	LinkedIn + Upwork blockchain outsourcing
Outcome	Understand wallet security and explain how blockchain accounts sign transactions

Safety rule for this lecture

Use only a new test-only wallet with no real funds. Never paste a real seed phrase or private key into websites, screenshots, chat, source code, Git repositories, or classroom examples. A blockchain professional treats signing secrets as production credentials.

20-Day Blockchain Developer & Freelancer Program

Day 2 at a Glance

Day 1 explained how a transaction travels through the network. Day 2 focuses on the cryptographic identity that creates and authorizes that transaction: keys, addresses, wallets and signatures.

Time	Lecture Segment	Teaching Goal
00-05	Day 1 bridge + wallet mental model	Separate blockchain state from wallet software
05-13	Private keys, public keys & addresses	Understand account identity
13-20	Digital signatures	Understand authorization without revealing secrets
20-27	Wallet types & account models	Software, hardware, custodial and smart accounts
27-35	Seed phrases & HD wallets	Understand recovery and deterministic derivation
35-43	Transaction & message signing	Review exactly what a wallet signs
43-50	Wallet security	Prevent common key-loss and phishing failures
50-56	Hands-on test-wallet lab	Inspect addresses and signing safely
56-58	Freelancer client scenarios	Apply knowledge to real job requests
58-60	Quiz, recap & homework	Check understanding and prepare Day 3

Day 2 success test

The learner should be able to explain: “A wallet does not contain coins. It manages signing credentials. The blockchain tracks state; the private key authorizes an EOA; the public key/address identify the account; and a digital signature proves authorization without revealing the private key.”

Learning Objectives

- Explain wallet vs account vs blockchain balance.
- Describe the relationship private key -> public key -> address.
- Explain why a digital signature is not encryption.
- Compare custodial, software, hardware and smart-contract wallets.
- Explain seed phrases and hierarchical deterministic (HD) wallet derivation.
- Distinguish transaction signing, message signing and typed-data signing.
- Recognize common wallet-security failures and safe professional practices.
- Handle client wallet issues without asking for secret recovery material.

00-05 min - Wallet Mental Model

Opening question

Ask: “If you delete MetaMask from your computer, did your ETH disappear from the blockchain?” The answer is no. The network state remains; what may be lost is convenient access to the signing secret if it was not backed up.

1. A wallet does not literally store your coins

On a public blockchain, balances and token ownership are recorded in blockchain state. Wallet software gives a user a way to create or import accounts, manage signing keys, display balances, connect to applications, build transactions and sign authorization requests.

```
Blockchain state: address -> balances / contract state
Wallet software: keys + account management + transaction signing + UI
```

What to say

“Your wallet is closer to a secure key manager and transaction signer than a bank account database. The assets are represented by the blockchain. The wallet proves that you are authorized to control an account.”

2. Four concepts to keep separate

Concept	What it is	Beginner mistake
Blockchain	Network + shared state/history	Thinking the wallet application is the blockchain
Account / Address	Identifier used in blockchain state	Thinking the address itself is secret
Private Key	Secret signing credential	Sharing it because it looks like an address
Wallet	Software/device/service that manages accounts and signing	Thinking coins are stored inside the app

Freelancer relevance

When a client says “my wallet is broken,” first determine whether the issue is the wallet UI, network/RPC, wrong chain, lost account import, contract interaction, or actual on-chain state. These are very different problems.

05-13 min - Private Keys, Public Keys & Addresses

3. The private key

For a normal Ethereum externally owned account (EOA), control begins with a private key. Conceptually, it is a randomly selected secret number in the valid range of the secp256k1 elliptic-curve system. The key must remain secret because possession of it is enough to create valid signatures for that account.

- Never send a private key to a client, support agent or developer.
- Never place a private key directly in frontend JavaScript.
- Never commit a private key or mnemonic to Git.
- Never use a production private key in a classroom demo.
- Treat exposed keys as compromised even if nobody admits using them.

Important

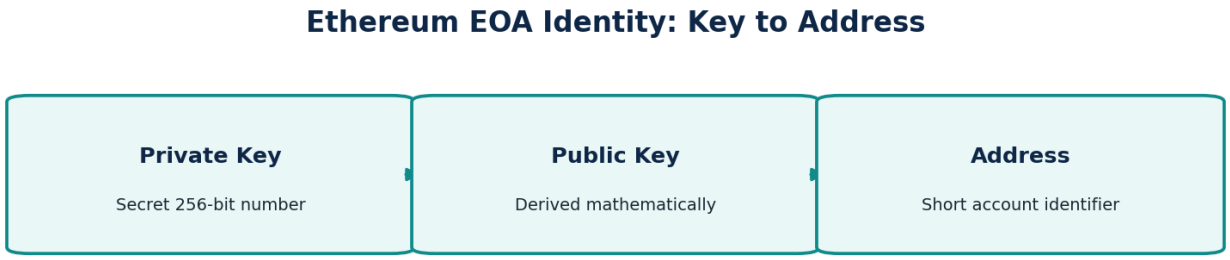
A private key is not a password that a website can reset. In self-custody there may be no central authority able to recover it for you.

4. Public key

The public key is mathematically derived from the private key. The derivation is designed to be easy in the forward direction but computationally infeasible to reverse. A public key can participate in signature verification without revealing the private key.

5. Ethereum address

An Ethereum EOA address is a 20-byte identifier derived from the public key. In practical developer work it is displayed as a 0x-prefixed hexadecimal string. Checksum capitalization such as EIP-55 can help catch typing errors in mixed-case addresses.



Do not oversimplify

Address derivation is not “private key hashed directly.” The private key derives the public key, then Ethereum derives the address from public-key material. This distinction matters when teaching cryptography accurately.

6. Address properties

- Public: safe to share when someone needs to send assets or inspect an account.
- Not an identity guarantee: an address does not inherently tell you the human or company behind it.
- Chain context matters: the same 0x address format can appear on multiple EVM networks with different balances/state.
- Contract addresses also use 20-byte address identifiers, but a contract account is controlled by code rather than a single private key.

13-20 min - Digital Signatures

7. What problem does a signature solve?

A blockchain needs a way to verify that an account holder authorized a transaction without requiring the private key to be sent across the network. Digital signatures solve this. The wallet signs a well-defined transaction or message digest using the private key. Other participants can verify the signature using public cryptographic information.

Digital Signature: Authorize Without Revealing the Private Key

Core idea

The private key creates the signature; the private key is not transmitted with the transaction. The signature is evidence of authorization, not a copy of the private key.

8. Signature vs encryption vs hashing

Technique	Main purpose	Can it be reversed?
Hashing	Produce deterministic digest / integrity fingerprint	Designed to be one-way
Encryption	Hide data from unauthorized readers	Yes, with the correct decryption key
Digital signature	Prove authorization/integrity of a specific message	Not “decrypted”; it is verified

9. What changes if the message changes?

A signature is tied to the exact data being signed. If the destination, amount, nonce, chain context or signed message changes, the original signature should no longer validate for the modified payload. This is why wallet screens must show users exactly what action they are authorizing.

Class question

If Alice signs “Send 1 ETH to Bob,” can an attacker change the signed transaction to “Send 100 ETH to Eve” and keep the same valid signature? No - changing signed data invalidates the signature.

20-27 min - Wallet Types & Account Models

10. Software wallet

A software wallet stores or accesses key material on a computer or phone and signs through application software. Browser-extension and mobile wallets are common examples. Convenience is high, but the device and browser environment become important security boundaries.

11. Hardware wallet

A hardware wallet is designed to keep signing keys isolated inside a dedicated device. The host computer prepares a transaction; the hardware device displays or receives transaction details and signs internally after user approval. The private key should not need to leave the device.

12. Custodial wallet/account

With custody, a service controls or co-controls the signing keys on behalf of the user. An exchange account is a common example. The user may authenticate with email, password, passkey or MFA, while the service operates blockchain wallets behind the scenes.

13. Smart-contract wallet / smart account

Not every blockchain account is controlled by one EOA private key. Smart-contract wallets can implement rules such as multisig authorization, spending limits, recovery mechanisms or account-abstraction features. The controlling logic is on-chain code.

Wallet model	Who/what signs or authorizes?	Typical advantage	Typical risk
Software self-custody	User-held key on device	Convenient and direct	Malware/phishing/device compromise
Hardware self-custody	Key isolated on hardware device	Strong key isolation	Bad confirmation habits / backup loss
Custodial	Service-managed keys	Easy recovery/user experience	Counterparty/custody risk
Smart-contract wallet	Contract-defined authorization rules	Flexible controls/recovery	Contract/configuration risk

Career note

In client discovery, ask what wallet model is used. “We use MetaMask,” “we use Fireblocks,” “we use Safe multisig,” and “we use exchange custody” imply very different integration and security requirements.

27-35 min - Seed Phrases & HD Wallets

14. Why seed phrases exist

Managing many unrelated private keys is difficult. Modern deterministic wallets commonly start from one backup secret and derive many accounts. A BIP-39 mnemonic is a human-readable representation of entropy plus checksum information that can be converted into a seed. That seed can feed hierarchical deterministic key derivation.

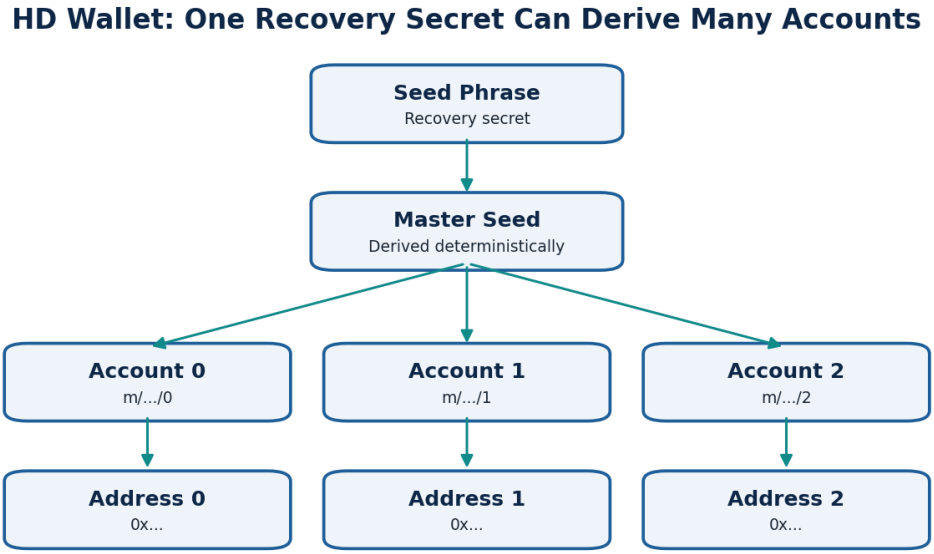
Important distinction

Seed phrase != wallet password. A local wallet password often encrypts/unlocks the wallet data on one device. The recovery phrase can recreate the deterministic wallet elsewhere and is therefore much more sensitive.

15. BIP-39 mnemonic basics

- Common mnemonic lengths include 12, 15, 18, 21 and 24 words.
- The words come from a defined 2048-word list for each supported language.
- A mnemonic encodes entropy plus checksum bits; it is not a random sentence chosen by the user.
- The optional BIP-39 passphrase changes the derived seed. Losing it can make the same mnemonic derive a different wallet than expected.

16. Hierarchical deterministic wallets



HD wallets derive child keys in a hierarchy. This means one backup can deterministically recreate many accounts. BIP-44 defines a common multi-account path structure. A widely used Ethereum path for the first account is:

```
m/44'/60'/0'/0/0
```

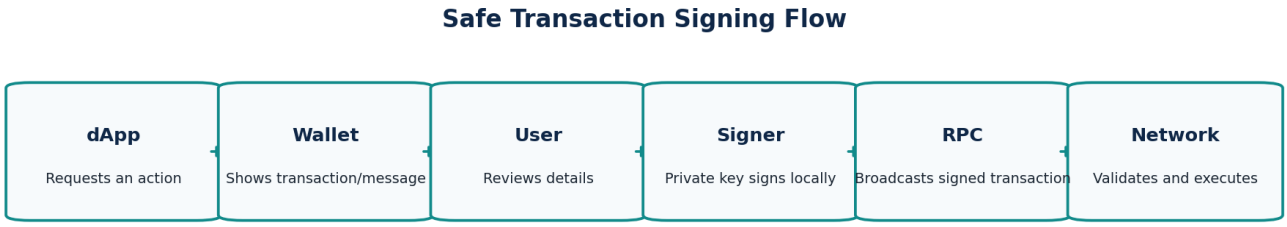
Path part	Meaning
44'	BIP-44 purpose
60'	Ethereum coin type
0'	Account index
0	External/change branch in the convention
0	Address index

Practical debugging

If a user restores the correct mnemonic but sees the “wrong address,” the issue may be a different derivation path, account index, BIP-39 passphrase, or wallet implementation - not necessarily a bad mnemonic.

35-43 min - Transaction & Message Signing

17. Transaction signing flow



An Ethereum transaction contains fields that define an intended state-changing action. Depending on transaction type, these include destination, value, calldata, nonce, gas parameters and chain identifier. The wallet serializes the transaction according to protocol rules and signs the appropriate digest.

Why chain ID matters

Chain context is part of modern Ethereum transaction signing. It helps protect against replaying a transaction on an unintended compatible chain. A professional should always verify the selected network before signing.

18. Three things a wallet may ask you to sign

Signing request	Purpose	Main caution
Blockchain transaction	Changes on-chain state or transfers value	Check chain, contract/to, value, gas and function
Plain/personal message	Proves control of an address or authorizes off-chain logic	A signature can still be security-sensitive
Typed structured data (EIP-712)	Human-structured signing for orders, permits, login/authorization flows	Understand domain, spender/operator, amount and expiry

19. “Sign” does not always mean “send transaction”

Message signing can happen without paying gas or changing chain state. This is useful for login challenges and off-chain orders, but it also means a malicious site can request a dangerous signature that later authorizes an action. Users should treat signatures as authorization, not harmless popups.

Instructor demo concept

Show two hypothetical wallet prompts: (A) “Sign in to example.app - nonce 4812” and (B) “Permit spender 0x... to use 1,000,000 USDC until ...”. Ask which one deserves closer review and why. Both should be understood before signing.

43-50 min - Wallet Security

20. Most wallet failures are operational, not cryptographic breaks

Attackers rarely need to “crack” modern elliptic-curve cryptography. It is usually easier to steal the recovery phrase, trick a user into signing, compromise a device, abuse token approvals, impersonate support staff, or exploit poor operational controls.

21. Common failure modes

- Phishing site asks to “verify” or “synchronize” a seed phrase.
- Malware reads clipboard content and replaces a copied destination address.
- User signs an approval/permit granting a malicious spender authority.
- Developer stores production keys in .env and accidentally commits the file to a public repository.
- Team shares one private key in chat instead of using role separation or multisig controls.
- User installs a fake wallet extension or downloads software from an unofficial source.
- Hardware-wallet user approves without checking the device display.
- User stores the only seed backup in an online screenshot or cloud note.

22. Professional security checklist

- Use test wallets and testnets for development.
- Keep production signing secrets out of source code and frontend bundles.
- Use environment/secret-management systems for backend keys when programmatic signing is truly required.
- Prefer hardware signing or institutional custody/multisig for significant funds.
- Verify destination, amount, network and contract before signing.
- Use small test transactions for unfamiliar workflows.
- Review token approvals and revoke unnecessary permissions.
- Keep recovery backups offline and physically protected.
- Never request a client seed phrase or raw private key just to troubleshoot an on-chain issue.

Client trust rule

A legitimate debugging engagement normally needs public information first: network, address, transaction hash, contract address, error, expected result and relevant application logs. Asking for seed phrases creates unnecessary security risk and professional liability.

50-56 min - Hands-On Test-Wallet Lab

Lab safety

Create a brand-new test-only wallet. Do not fund it with real assets. Do not display or submit its seed phrase/private key anywhere in the exercise.

23. Exercise A - Create and inspect a test account

1. Install/open a reputable wallet from its official distribution channel.
2. Create a new wallet intended only for training.
3. Write the recovery phrase privately on temporary offline paper for the exercise; never paste it into this document or chat.
4. Record only the public address in your notes.
5. Switch to a supported Ethereum test network and confirm the network name/chain context.
6. Open a blockchain explorer and search your public address.

Expected observation

A newly created address can exist cryptographically before it has any transaction history. An explorer may show no balance or transactions until the account interacts with the chain.

24. Exercise B - Sign a harmless test message

If your wallet/application environment offers a clearly understood test message-signing feature, sign a harmless string such as:

```
Blockchain Day 2 training - test account only
```

Observe that signing a message does not inherently broadcast a state-changing blockchain transaction. The signature proves that the controlling key authorized that exact message. Do not sign random text from unknown websites.

25. Student worksheet

Question	Your answer
Public test address	
Network selected	
Did the address already exist before funding?	
What secret must never be shared?	
Did message signing alone create an on-chain transaction?	

56-58 min - Freelancer Client Scenarios

Scenario 1 - “I restored my wallet but the address is different.”

Investigate before concluding that funds are lost. Ask which wallet software was used, which chain, whether an optional mnemonic passphrase existed, which account index was expected, and whether a custom derivation path was used. Never ask the client to send you the seed phrase.

Scenario 2 - “My dApp needs users to prove they own an address.”

A common architecture is challenge-message authentication: the backend generates a unique nonce, the wallet signs a human-readable message, and the backend verifies the signature and expected address. The challenge should be unique and expire so signatures cannot be reused indefinitely.

Scenario 3 - “Our frontend contains the deployer private key.”

This is a critical design problem. Anything shipped to a browser must be treated as public. Move privileged signing to an appropriate backend/secure signer, or redesign so users sign their own transactions and administrative actions use secure operational controls.

Scenario 4 - “Customer signed something and assets were drained.”

Collect public evidence: wallet address, transaction hashes, token contracts, approval/permit transactions, spender/operator addresses and timestamps. Determine whether value was transferred directly, an allowance was used, or a signature-based authorization was exercised.

Upwork-ready discovery questions

Which network? Which public address? Which transaction/signature? Which wallet type? What action did the user intend? What was displayed in the wallet prompt? Which contract/spender was involved? What is the expected result?

58-60 min - Mini Quiz & Recap

26. Ten rapid questions

- Where are ETH/token balances represented: inside MetaMask or on the blockchain?
- Which item must remain secret: address, public key or private key?
- Can a digital signature be verified without revealing the private key?
- Is hashing the same as encryption?
- What is the main difference between a wallet password and a seed phrase?
- Why can one HD wallet produce many addresses?
- What does a hardware wallet try to keep isolated?
- Does signing a message always create an on-chain transaction?
- Why should a developer check the selected chain before signing?
- What public information should you ask a client for before ever discussing secrets?

Answer key

1) Blockchain state. 2) Private key. 3) Yes. 4) No. 5) Password commonly unlocks/encrypts local wallet data; seed phrase can recreate deterministic keys. 6) Deterministic child-key derivation. 7) Private signing keys. 8) No. 9) To prevent unintended actions/replay and ensure correct state. 10) Network, public address, transaction hash, contract address, expected result and logs.

27. One-minute verbal recap

```
Private Key -> Public Key -> Address
Wallet -> manages keys/accounts and signing
Signature -> proves authorization without revealing secret
Seed Phrase -> recovery material for deterministic wallet
HD Wallet -> derives many child keys/accounts
Blockchain -> holds the shared state and transaction history
```

Day 2 completion standard

The learner passes Day 2 when they can explain the full key/signing model in their own words and can troubleshoot wallet questions without requesting or exposing secret recovery material.

Day 2 Reference Notes

28. Ethereum EOA technical model - beginner-safe depth

- Ethereum EOAs use the secp256k1 elliptic curve for ECDSA signatures.
- A private key is effectively a 256-bit secret scalar within the curve order.
- The public key is an elliptic-curve point derived by scalar multiplication.
- Ethereum derives a 20-byte EOA address from the Keccak-256 hash of the uncompressed public key data (excluding the 0x04 format prefix), using the rightmost 20 bytes.
- Addresses are usually shown as 40 hexadecimal characters plus the 0x prefix; mixed-case checksum encoding can detect many typing mistakes.

Teaching note

Do not require beginners to perform elliptic-curve mathematics on Day 2. They should understand the direction of derivation, the one-way security property, and why signatures can be verified publicly.

29. Nonce and transaction authorization

For EOAs, the transaction nonce orders outgoing transactions and prevents the same signed account transaction from being accepted repeatedly in the normal transaction sequence. Day 3 will examine account state, nonce, gas, calldata, receipts and the EVM in more detail.

30. Signature recovery intuition

Ethereum signatures include enough information for protocol/client logic to recover the signing public key/address from the signed digest. Modern libraries usually hide the mathematical details. As a developer, you will use library functions to sign and verify rather than implement ECDSA yourself.

31. Smart-contract accounts are different

A contract address has no ordinary EOA private key. Its behavior is defined by code. Smart-account systems can validate signatures or other authorization methods inside contract logic. This distinction becomes important when implementing multisig wallets, account abstraction and contract-based authorization.

Beginner Mistakes to Eliminate

Mistake	Correct mental model
“My coins are inside my wallet app.”	The wallet displays/manages access; the chain records ownership/state.
“My wallet password can recover my blockchain account anywhere.”	Usually the recovery phrase/private keys matter for recovery; local passwords are device/app-specific.
“My address must stay secret.”	Public addresses are designed to be shared; the private key/seed phrase must stay secret.
“A signature is harmless because it costs no gas.”	Off-chain signatures can authorize important actions such as permits/orders/login. Read what you sign.
“A hardware wallet makes every transaction safe.”	It protects key isolation, but users can still approve malicious or incorrect transactions.
“If I know the public key, I can calculate the private key.”	Modern cryptography relies on that reverse computation being infeasible.
“Every 0x address is a user wallet.”	It may be an EOA or smart contract address.
“Support needs my seed phrase to debug.”	Legitimate troubleshooting normally starts with public on-chain data. Never disclose recovery secrets.

Day 2 Glossary

Term	Meaning
EOA	Externally owned account controlled by a private key.
Private key	Secret cryptographic signing credential.
Public key	Public cryptographic value derived from the private key.
Address	Short blockchain identifier used in account/state interactions.
Wallet	Software, device or service managing accounts and signing.
Signature	Cryptographic proof that a specific key authorized specific data.
Mnemonic / seed phrase	Human-readable recovery representation used to derive wallet seed material.
HD wallet	Hierarchical deterministic wallet that derives many keys/accounts.
Derivation path	Path that selects a child key/account in an HD hierarchy.
Custody	Who controls the signing keys.
EIP-712	Ethereum standard for typed structured data signing.
Multisig	Authorization requiring multiple signers/keys under defined rules.

Day 2 Homework & Portfolio Practice

Homework A - Explain the account model

Record a 90-second explanation using the following sequence without reading your notes:

```
private key -> public key -> address -> wallet -> signature -> transaction -> blockchain
```

Homework B - Wallet comparison

Create a one-page comparison of software wallet, hardware wallet, custodial wallet and smart-contract wallet. For each, write who controls signing, how recovery works, and one security risk.

Homework C - Test account inspection

- Use only the Day 2 test wallet with no real assets.
- Find its public address on a testnet explorer.
- Record the address and chain name.
- If you perform a test transaction later, record its hash - never the private key or seed phrase.

Homework D - Security incident response

Write your response to this fictional client: “I entered my seed phrase into a website and now I am not sure whether it was legitimate.” Your response should prioritize moving remaining assets using a trusted environment/new wallet, avoiding further signing from the compromised wallet, preserving transaction evidence, and never reusing the exposed secret. Do not promise that compromised credentials can be made secret again.

Homework E - Interview preparation

- What is the difference between a private key and seed phrase?
- What does a digital signature prove?
- What is an HD wallet?
- What is a derivation path?
- Why is a hardware wallet safer than storing a raw key in source code?
- What is the difference between signing a transaction and signing a message?

Prepare for Day 3

Day 3: Ethereum & EVM - EOAs vs contract accounts, account state, nonce, gas, calldata, bytecode, ABI, logs/events, transaction receipts and how a smart-contract call actually executes.

Printable Day 2 Cheat Sheet

```
WALLET MODEL
Blockchain = shared balances/state/history
Wallet = account/key manager + signer + UI

EOA IDENTITY
Private Key -> Public Key -> Address
SECRET         PUBLIC       PUBLIC

SIGNING
Data -> Digest -> Sign with private key -> Signature -> Verify

HD WALLET
Mnemonic -> Seed -> Master Key -> Child Keys -> Addresses
Example Ethereum path: m/44'/60'/0'/0/0

DO NOT SHARE
Private key | Seed phrase | Hardware-wallet recovery backup

SAFE TO SHARE FOR DEBUGGING
Network | Public address | Transaction hash | Contract address | Error/result

CHECK BEFORE SIGNING
Network | To/Contract | Value | Function | Spender/Operator | Amount | Expiry
```

Fast Freelancer Checklist

- Ask for public evidence first.
- Never request a seed phrase for routine debugging.
- Identify wallet model and network.
- Verify whether the issue is on-chain state or wallet UI/RPC.
- For signing problems, inspect the exact transaction/message/typed data.
- For lost-address issues, investigate derivation path/account index/passphrase.
- For compromised credentials, treat the secret as compromised and rotate/migrate rather than trying to “hide it again.”
- Use test wallets for development and teaching.

Remember

A blockchain developer does not merely know how to connect MetaMask. A professional understands what the wallet is authorizing, which key/account model is involved, what the network will verify, and how to protect client signing authority.